

Attention answers: what should interact?

Start with the information the bot needs. Then choose how to compute the interactions.



Your model has **spatial attention and temporal attention**. DeepSeek’s long-context techniques mostly target the second kind. A state-only chess model can retain substantial attention over squares.

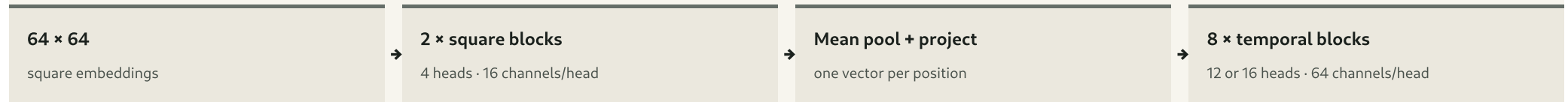
▼ Explain further

A ply is one player’s move. A square token describes one square; a temporal token describes a whole position. Neither is a generated language token. Search evaluates alternative future boards using legal transitions; temporal attention only reads the history of the branch being evaluated.

Source: [Code map](#)

The actual model has two attention stacks

Two small board layers feed eight much wider HSTU-style layers.



The board encoder uses bidirectional SDPA plus an MLP in each block. It pools the 64 outputs and projects to the trunk width. The trunk sees a sequence of these position vectors with metadata added.

ckpt34 lineage: D=768, H=12, L=8. **v4 config:** D=1024, H=16, L=8. Both use $d_k=d_v=64$. A config choice is not evidence that a particular checkpoint was trained with it.

▼ Explain further

The loader constructs the model from the chosen repository config, then loads weights strictly. It does not infer all architecture settings from the checkpoint. The board encoder's two 64-wide layers are hard-coded in `hstu_model.py`. Its cached outputs are not a temporal KV cache.

Source: [Code map](#)

One HSTU-style block, decomposed

SiLU prepares the projected features. Softmax mixes information across positions.

```
[U,V,Q,K] = split(SiLU(Wuvqk LN(X)))  
A = softmax(QKT / √dk + relative_bias + mask)  
X' = X + WO dropout(U ⊙ LN(concat_heads(AV)))
```

Q asks what this position needs. **K** makes another position addressable. **V** carries the information retrieved. **U** gates the retrieved result channel by channel. The residual addition preserves the input while adding an update.

The original HSTU paper uses a pointwise SiLU attention weighting scheme. This repository instead normalizes attention with **softmax**; its SiLU is on the UVQK projection. Those are different operations.

▼ Explain further

Here LN is layer normalization, $\odot$ is elementwise multiplication, and rows of A sum to one over permitted keys. Trunk normalizations use functional layer_norm without trainable affine parameters. The block has no separate conventional Transformer FFN; its projection, gating and output layer provide the pointwise transformation. Dropout acts on the gated output, not on attention probabilities; both main configs set it to zero.

Source: [Code map](#) · [HSTU paper](#)

Layers refine features; heads offer different views

Every temporal layer has its own projections, relative bias and K/V cache.



Layer 2 builds its Q/K/V from Layer 1's output. It can therefore compare features that already combine earlier information. Heads operate in parallel inside a layer; the output projection mixes their contributions.

There is no enforced assignment such as "head 3 finds forks." These are learned representations. **More layers do not mean more chess search plies.**

▼ Explain further

All previous tokens are visible in one causal layer; depth adds composition, not additional lookback in this full-attention model. Replacing Layer 2's cache with Layer 1's cache changes the function. Sharing position-index scratch across layers is already safe because it is layer-invariant; sharing learned K/V is an architectural change.

Source: [Code map](#)

The mask decides who may communicate

Packed games share a tensor but cannot read one another.

```
allowed(q,k) = same_game(q,k) AND k ≤ q
```

Game A: 0 1 2 3

→ **Game B: 0 1 2**

→ **Game C: 0 1 2 3 4**

CUDA dataset batches use a document-aware causal BlockMask. Absolute position embeddings restart at zero for each game. Relative bias is a learned table indexed by clamped key-minus-query distance, separately for every head and layer.

Training predicts the next move at each position simultaneously. The causal mask prevents future-board leakage into that prediction.

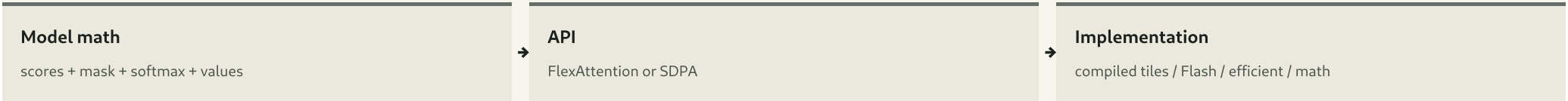
▼ Explain further

Positions are scaled content plus a learned absolute embedding; the repository does not use RoPE. Within a game, subtracting packed token indices gives the same distance as subtracting local indices. Across games, the mask blocks attention. Block sparsity skips fully masked tiles; boundary tiles can still perform partially wasted work.

Source: [Code map](#)

FlexAttention, SDPA and FlashAttention are different names

Separate the mathematical operation, its API, and the GPU kernel that executes it.



FlexAttention provides programmable score modification and block masking. Compiled execution can fuse the relative bias and avoid materializing the full score grid.

SDPA is PyTorch’s scaled-dot-product-attention API. Backend selection depends on dtype, device, masks and shapes. An SDPA call does not establish that FlashAttention ran.

▼ Explain further

FlashAttention reorganizes exact attention into memory-efficient tiles; it does not compress the model’s persistent KV representation. FlexAttention can express this repository’s attention without changing its meaning. Hardware support and backend rules vary with PyTorch versions; inspect the actual trace.

Source: [FlexAttention](#) · [SDPA](#)

The runtime map is broader than train versus eval

The mask type, workload shape and entry point determine the temporal attention route.

Workload	Temporal attention
Supervised CUDA training / dataset evaluation	BlockMask → FlexAttention
Stage-2 CUDA learning	BlockMask → eager FlexAttention in current trainer
Small root / full-prefix evaluation	Dense mask + per-layer bias → SDPA
Stockfish cached search	Manual prefix/suffix attention via decode methods
Default CUDA self-play collection	Whole TensorDecoder compiled; manual one-query attention
Benchmark SDPA decoder variants	Reusable workspace + SDPA; optional compilation

The square encoder uses SDPA in every one of these workloads. `.eval()` alone does not select the temporal backend. The Stockfish evaluator’s general compile flag also does not select the self-play TensorDecoder runner.

▼ Explain further

`scripts/train.py` explicitly compiles the model when configured. `Stage2Trainer` does not compile its forward/backward or enter autocast in the current path; compiling mask creation alone does not compile attention. CUDA collection defaults to the compiled TensorDecoder; its `sdpa` flag is false. Stockfish’s shared executors default to current/manual decode. CPU paths use dense SDPA for backward compatibility.

Source: [Code map](#)

Prefill and decode spend memory differently

Prefill processes many queries. Cached decode processes only a new position.



For a branch leaf, cached decode needs only a score row per head. It does not re-run the board encoder for every ancestor. The code combines all score parts before one softmax; separately normalizing prefix and suffix would change the answer.

Sibling branches can share immutable prefix storage. They must not attend to each other's imagined futures.

▼ Explain further

Causal attention makes earlier K/V stable when later tokens are appended. Current search stores new K/V per layer, uses an arena for branch entries, and gathers needed ancestor suffixes. Packed prefixes and temporary suffix buffers can consume additional memory. Reusing stored bytes is not the same as avoiding their reads on each query. Neural caches must be invalidated when model weights change.

Source: [Code map](#)

There are four separate memory bills

A smaller KV cache does not automatically fix training memory.



Dense additive mask per layer = $H \times S^2 \times \text{bytes}$

Temporal KV bytes = $L \times T \times H \times (d_k + d_v) \times \text{bytes}$

S is total packed batch tokens; T is one game’s cached positions. For v4, H=16 and S=4096, an FP32 additive mask alone is **1 GiB**. The layer guard refuses masks larger than 256 MiB.

▼ Explain further

Fused attention avoids storing a complete S×S score matrix; it still stores activations and does arithmetic on permitted token pairs. In a packed batch the useful temporal work scales with the sum of squared game lengths, not the square of all games concatenated. The dense route explicitly constructs the large bias even if SDPA itself is fused. The guard does not cover every temporary allocation.

Source: [Code map](#)

Compilation matters for self-play learning

The current stage-2 trainer invokes FlexAttention without compiling the model.

Compiled BlockMask builder



Eager model forward



Unfused attention + backward



Large memory growth

Compiling the mask builder only constructs mask metadata. It does not fuse the attention call inside the training model. This is a concrete execution issue to investigate before importing a new model architecture.

A recorded 3070 Ti replay probe completed 1,024-token batches with **2.725 GB peak allocated**; 2,048 tokens ran out of memory. This is historical evidence, not a fresh benchmark of today's environment.

▼ Explain further

Local PyTorch at this review is 2.14.0+cu130. Repository comments mention earlier 2.13 constraints; recheck them before changes. A bounded experiment should compile a disposable stage-2 model or its attention region, verify gradients against eager, then measure warmed forward+backward and peak allocations on the same replay. Board encoder activations and losses still consume memory. No new training benchmark was run for this guide.

Source: [Code map](#) · [Recorded replay probe](#)

Latency: use the measurements we actually have

On the recorded collection workload, compilation mattered more than switching decode to SDPA.

Decoder	Trial seconds	Eligible positions/hour	Peak GB
Eager	272.55 / 276.00	36,319	0.966
Compiled manual	226.64 / 219.13	44,705	0.966
Compiled SDPA	234.46 / 205.07	45,530	1.113

RTX 3070 Ti Laptop, FP32, TF32 disabled, ckpt34, G=24, 128 simulations, 16 candidates, depth cap 32. Each trial collected 32 games. Disk compilation caches were warm; model loading was excluded.

The SDPA variant used the memory-efficient CUTLASS backend. Its ~1.85% incremental throughput gain was variable and below the recorded adoption threshold.

▼ Explain further

These are whole-collection times, not isolated attention latency, training speed or Elo. Only two final trials per listed mode are available. Compiled SDPA also had a 0.001105 target-probability outlier in the campaign. The source report distinguishes root-output agreement from proof of identical interior search decisions. A newer GPU or BF16 could change the result.

Source: [Benchmark report](#) · [Raw summary](#)

Start the DeepSeek transformation with ordinary MHA

Many query heads normally keep many key/value representations per token and layer.



There are three independent places to shrink this storage: **channels per entry**, **entries across time**, and **independent cache owners across layers**. Precision supplies another multiplier.

Separately, sparsity changes which stored entries a query reads. Selecting fewer entries does not by itself delete the unselected cache.

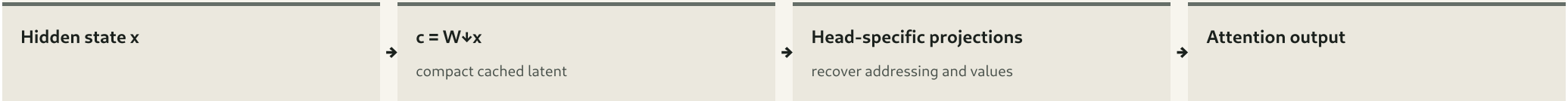
▼ Explain further

This decomposition is a teaching path through ideas, not a claim that DeepSeek-V4.1 is literally an HSTU block with several switches enabled. The model family also changes normalization, positional encodings, output projections, residual connections and feed-forward architecture.

Source: [Local report](#) · §2.3

MLA compresses the representation of each entry

Multi-head Latent Attention keeps a shared latent rather than expanded K/V for every head.



For linear projections, $q^T(W_K c) = (W_K^T q)^T c$, so inference can move work to the query side without storing expanded keys. Value/output projections can likewise be combined. DeepSeek-V2 uses a separate positional component to accommodate RoPE.

Chess hypothesis: smaller branch caches could support more concurrent searches. This does not itself improve tactical accuracy.

▼ Explain further

This is the MLA idea introduced with V2, not the full V4.1 architecture. The current HSTU block applies SiLU to K/V projections, so linear absorption is not an exact drop-in rewrite. An MLA-style fork requires redesigned projections and training. Re-expanding all K/V each step can erase the bandwidth benefit.

Source: [DeepSeek-V2 §2.1](#)

GQA offers a simpler channel-sharing experiment

Keep many query heads, but let groups of them share fewer K/V heads.



The query heads can still ask different questions. Their addressable representations share capacity. This is a simpler architectural fork than MLA, but the quality tradeoff must be learned and measured.

For chess, test it if cache memory or bandwidth limits batching. A smaller stored cache can still be slow if the kernel expands it or poorly handles the head grouping.

▼ Explain further

The 4× factor is arithmetic for 16→4 KV heads at fixed dimensions and dtype, not a measured speedup. The repository currently uses one K/V head per query head. A change affects training projections, every cache layout and decode path, and checkpoint compatibility.

Source: [GQA paper](#) · [Code map](#)

Sparse attention selects which memories to read

A lightweight indexer chooses candidates; the main attention reads that subset.



Indexing costs work too. It is useful when selecting a small subset avoids much larger full attention. Selection can miss important information, so the model must learn under the sparse mechanism.

DeepSeek-V4.1 keeps up to **512 global entries** per query. With ≤ 513 temporal positions in this chess model, that choice offers little temporal sparsity.

▼ Explain further

“Space attention” is ambiguous. In this report the relevant term is sparse attention. Spatial attention means interactions over board squares and already exists in this bot. Chess search’s top_m=16 candidate moves and DeepSeek’s Top-512 attention entries select entirely different objects. The former searches legal actions; the latter retrieves representations.

Source: Local report · §§2.3, 4.2.1

Compress time, keep a local window

A global branch summarizes older entries; local sliding-window attention preserves recent detail.



V4 used CSA and heavily compressed attention (HCA). V4.1 uses CSA2: its encoder compresses with $m=2$, while its decoder uses $m=1$. CSA2 removes the earlier compressor's overlapping groups and absolute position embedding within compression.

Every V4.1 layer also has a local window of 128 tokens; the first two layers are local-only.

▼ Explain further

The global selection and local keys are concatenated for the main attention; this is not two independently normalized answers simply added together. In chess, adjacent plies alternate players and can contain a decisive capture: averaging or learned compression can obscure important distinctions. Do not assume token compression preserves tactics. Sliding temporal windows also differ from limiting attention to nearby squares.

Source: [Local report](#) · §§2.1, 2.3, 4.2.1

CSA2 shares storage across layers

Full, Reindex and Reuse describe who produces the global cache and the selected indices.

Mode	Main KV / indexer K	Selected IDs	Always fresh
Full	Produces	Computes	Main Q + local KV
Reindex	Shares previous Full	Computes again	Main Q + local KV
Reuse	Shares previous Full	Shares latest selection	Main Q + local KV

Sharing K/V saves storage. Sharing index IDs saves selection work. **Neither shares the final attention output:** each layer still uses a fresh main query and computes a new update.

▼ Explain further

In V4.1’s encoder, the 18 CSA2 layers form three groups of six: Full + five Reuse layers. In its 20-layer decoder, there is one Full group then four Reindex groups, each followed by three Reuse layers. Thus three encoder global-cache owners plus one decoder owner serve 38 CSA2 layers. All 40 layers retain local processing. This preserves more layer-specific computation than simply tying every projection.

Source: [Local report](#) · Fig. 4 and §4.2.1

CED changes where upper-layer memory comes from

Decoder global K/V is projected from the final causal encoder state.



Ordinary decoding constructs each layer’s K/V from that layer’s input. CED makes the upper global cache available from the encoder output, allowing most long-prompt tokens to bypass full decoder computation during prefill.

Generation still runs encoder and decoder. Local decoder K/V still depends on local decoder states and needs replay.

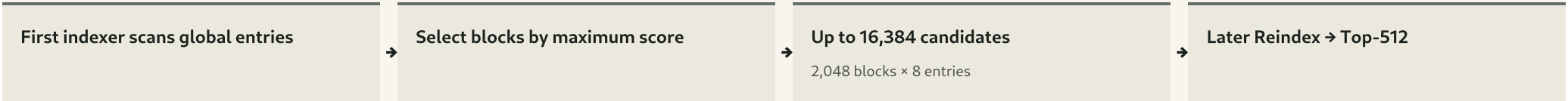
▼ Explain further

The report gives approximately $N \cdot L/2 + W \cdot L/2$ prefill work for $N \gg W$, versus $N \cdot L$, under bounded decoder replay. This does not promise to halve per-move latency in chess. The recorded chess collection campaign attributed only ~2.5–3% of whole collection to the root executor: saving half of a 3% component saves ~1.5% overall, if everything else stays equal.

Source: [Local report](#) · §2.2 · [Chess profile context](#)

Hierarchical indexing reuses the search domain

Later indexers search a candidate pool built by the first decoder Full layer.



Each later Reindex layer can choose a different Top-512 within the shared pool. Reuse layers skip indexing. The candidate restriction is introduced during post-training as well as inference.

The first indexer still scans the causally visible range. The technique bounds later indexing work; it does not make all context-dependent work disappear.

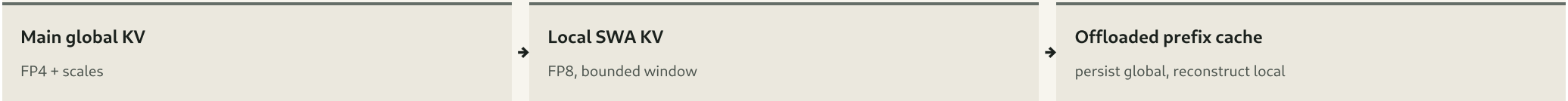
▼ Explain further

A 16,384-entry pool is far larger than a chess game here. Reducing those constants is a new design requiring evidence, not a ready-made speed improvement. For board-square attention, selecting only a few squares could miss a distant bishop or rook. At 64 tokens, routing overhead may exceed the arithmetic saved.

Source: [Local report](#) · §2.3.2 and Fig. 5

FP4 shrinks bytes; bounded replay shrinks persistence

These solve different storage problems and introduce different approximations.



V4.1’s main KV uses E2M1 values plus one E4M3 scale per 16 channels: nominally **4.5 bits/channel**. Values are dequantized for attention. Local SWA stays FP8 because it is more sensitive.

Bounded replay reconstructs local state from the most recent 128 tokens instead of the full dependency span. The report explicitly says reconstructed states are **not mathematically identical**.

▼ Explain further

Global KV means main KV plus indexer K spanning the context; it does not mean a cache shared across unrelated requests. Persistent KV means the subset saved for later prefix reuse in host memory/SSD. The headline 890 bytes/token is a whole-model global-cache figure, not total serving memory; local windows, weights and workspace still exist. Quantization-aware training and replay adaptation support these approximations. Chess would need validation near draw boundaries and sharp tactics.

Source: [Local report](#) · §§1, 2.4.4, 3.2.2

Other V4.1 components are not attention compression

The report also changes residual mixing, memory lookup and token generation.

Component	What it does	Chess interpretation
Single-Pass mHC	Mixes multiple residual streams; shifts mixing coefficients to enable fusion	Possible depth/optimization experiment, not a KV-sharing switch
Engram	Conditional hashed n-gram memory	Learned lookup; unlike an exact transposition table or tablebase
DSpark	Drafts tokens and schedules verification	Move proposals still require legal transitions and adversarial search
MoE	Routes tokens through selected feed-forward experts	Capacity idea; routing overhead matters at small batches

These can complement attention changes, but none establishes a strength or speed benefit for this bot. Language-token verification is not equivalent to verifying that a chess line is strategically sound.

▼ Explain further

Single-Pass mHC uses the previous block’s input-mixing coefficients to eliminate a dependency and reduce activation traffic in a fused deployment kernel. Engram’s learned memory addresses token n-grams, not complete chess-rule states. DSpark’s separate drafter predicts five positions with a lightweight dependency model; copying that process would not replace minimax-like adversarial reasoning.

Source: [Local report](#) · §§2.4.1–2.4.3

A board is almost enough; rule state finishes the job

Chess is Markov when “state” includes the information required by its rules.



A position diagram alone is incomplete. Even FEN contains no repetition record. Two histories can end with the **same complete FEN and previous move**, but only one permits a threefold claim. Try the verified example below.

The current model includes castling, en passant, bucketed halfmove/fullmove counts and previous move. It has no explicit repetition feature; the engine tracks terminal rules separately.

▼ Explain further

The count of the current position alone is not an exact sufficient statistic for future repetition decisions: the engine needs occurrence information for other reachable prior positions too. A repetition-key/count map since the relevant irreversible boundary can support exact rule tracking. A small learned feature is an approximation; terminal handling in search remains exact. Fifty-/seventy-five-move thresholds, repetition claims and automatic draws must stay distinct. Fullmove number is not intrinsically needed for chess value, although it can carry dataset artifacts.

Source: [Code map](#) · [FIDE arts. 9.2–9.6](#)

History can help a trained model without being necessary

Sufficient information and sufficient learned computation are different questions.



History might help infer repetition, compensate for a pooled board representation, or reveal move plans. It might also encourage style and outcome shortcuts, redundant computation and path-dependent value estimates.

None of those effects proves that history is harmful or beneficial here. Deleting history at evaluation tests robustness to an input change; it does not fairly test a model trained to use only state.

▼ Explain further

Masking history at every layer leaves a single temporal key, so softmax becomes exactly 1. Q/K then have no effect on mixing; the gated V and residual transformations remain. Use a purpose-built state-only architecture for the serious experiment. For a short per-layer window, higher-layer cached states can carry older information indirectly: this differs from recomputing a hard last-k input segment.

Source: [Code map](#) · Hypotheses, not observed strength results

The most useful fork may spend depth on the board

Keep interactions between squares alive longer instead of pooling after two layers.



Our hypothesis: richer within-board reasoning could use compute more directly than repeated attention over old pooled boards. A rook and king may matter across the entire board; full 64-square attention is an inexpensive baseline in sequence length, though width and depth still cost real compute.

Compare to a small gated MLP on the pooled board and to short-history HSTU. A state-only model can make exact input-equivalent neural results reusable across search transpositions.

▼ Explain further

The existing joint piece-square embedding already encodes placement; a geometric bias would add an inductive preference rather than recover absent coordinates. Reusing only the board-encoder output for identical piece placements is exact for fixed weights in evaluation, even if metadata differs; cache lookup cost must pay off. Full-result caches must include every actual model input and a weight version. History-dependent current models cannot safely use board-only keys for their whole-network output.

Source: [Code map](#) · Proposed experiments

An experiment order that can answer the chess question

Separate execution efficiency, representation quality and search strength.

1. **Audit stage-2 fusion.** Same weights, same replay, gradients and warmed forward/backward memory.
2. **Run matched history ablations.** Current-only, short context (e.g. 4 or 16), full history; train for each input contract.
3. **Strengthen the state representation.** Explicit rule features, then board depth/readout experiments.
4. **If temporal memory still pays:** try fewer KV heads, then latent or cross-layer sharing.

Evaluate policy quality, value calibration and tactics first; then paired chess matches. Use both **fixed neural-evaluation budgets** to compare decision quality and **fixed wall-clock budgets** to compare practical engines.

▼ Explain further

Hold data, game-level splits, seeds, optimizer schedule, search policy and Stockfish protocol constant; separately report parameter/FLOP differences and exposure counts. Repeated positions and transpositions need leakage-aware splits. Report uncertainty over enough paired games. Track total game runtime, model time, positions/hour, peak allocated/reserved memory, and cold/warm compilation separately. Measure attention's fraction before predicting speedups. No architecture change or new strength claim is part of this guide.

Source: [Code map](#)